HACKATHON SQUAD

Maximum Weight Independent Set — Technical Report

IIT Guwahati Coding Club • Even Semester Projects 2026

1. Problem Statement

The Hackathon Squad problem asks us to assemble a conflict-free team of coders that maximises total skill rating. Each coder has a skill value, and certain pairs refuse to work together (conflict pairs). We must select a subset where no two members conflict, maximising the sum of their skill ratings.

Formally, this is the **Maximum Weight Independent Set (MWIS)** problem on an undirected graph:

- N nodes (coders), each with weight skill[i] — $(1 \le skill[i] \le 10^9)$
- M edges (conflict pairs) between coders
- Find $S \subseteq V$ such that no two nodes in S are adjacent, maximising Σ skill[i] for $i \in S$

Constraints: $1 \le N \le 200{,}000$ • $0 \le M \le N \cdot (N-1)/2$ • $1 \le skill[i] \le 10^9$ • 5-minute execution limit

Solutions are judged on two criteria: validity (no two selected coders conflict) and score (higher total skill relative to the best known solution). An invalid team is disqualified regardless of score.

---

2. Why This Problem Is Hard

MWIS is NP-hard, proven by polynomial reduction from Maximum Independent Set (MIS), which is NP-complete. Setting all weights to 1 in MWIS gives exactly MIS, so any polynomial MWIS solver would solve MIS — contradicting NP-completeness.

Practical consequences for N = 200,000:

- Exact methods (branch-and-bound, integer programming) are computationally infeasible.
- Brute-force enumeration of all 2^n subsets is astronomically impossible.
- The only viable approach is heuristics and metaheuristics that find high-quality approximate solutions within the time budget.

---

3. Data Structures

3.1 Core: Adjacency List

The graph is represented as an adjacency list — the standard and most efficient structure for sparse graphs:

```
vector<vector<int>> adj;   // adj[u] = list of coders who conflict with u
```

Space: O(N + M). Neighbour iteration: O(degree). This encodes all conflict relationships and is the foundation everything else is built on.

3.2 Supporting Optimisation: conflictCnt[]

On top of the adjacency list, a counter array is maintained to make feasibility checks O(1):

```
conflictCnt[u] = number of currently selected neighbours of u
canAdd(u)  ⇔  !inSet[u] && conflictCnt[u] == 0      // O(1) check
```

Without this, every canAdd() call would require scanning all neighbours: O(degree). With it, the check is a single integer comparison. Add/remove still run in O(degree) to update the counters, but each node is added or removed at most once per greedy pass.

Check / Operation	Without conflictCnt	With conflictCnt
canAdd(u)	O(degree) — scan all neighbours	O(1) — read one integer
add(u)	O(degree)	O(degree) — update neighbours
remove(u)	O(degree)	O(degree) — update neighbours

Key point: conflictCnt[] is a performance detail layered on top of the adjacency list. The adjacency list is the main data structure; conflictCnt[] just makes one specific operation faster.

---

4. Algorithm Design

The solution runs in two phases within the 5-minute window:

Phase 1 — Greedy Seeding (0 to 60 s)

Three greedy passes are run, each using a different node ordering. The best solution found is kept as the starting point for Phase 2.

Ordering	Sort Key	Rationale
A — Ratio	skill[u] / (degree[u] + 1) descending	Balances value against conflict cost
B — Skill	skill[u] descending	Greedy on pure value
C — Degree	degree[u] ascending	Low-conflict nodes first; fills value after

Each greedy pass runs greedyBuild() (add nodes in order if conflict-free) followed by fillFreeNodes() (one O(N+M) scan to add any remaining conflict-free nodes the ordered pass may have skipped), then a full 1-swap local search.

Phase 1 Detail — 1-Swap Local Search

After each greedy seed, the solution is improved by iterating over selected nodes (cheapest first) and trying to swap each out for its newly-freed neighbours:

- Remove node u from the set.
- Collect u's neighbours that now have conflictCnt == 0 (freed by u's removal).
- Greedily add those free neighbours in skill-descending order.
- Keep the swap if total gained > skill[u]; otherwise revert.
- After every pass, run fillFreeNodes() to catch any additionally freed nodes.
- Repeat until a full pass makes no improvement.

Time per pass: O(N · avg_degree²). Fast on sparse graphs; converges in a few passes on most inputs.

Phase 2 — Iterated Greedy (60 s to 275 s)

1-swap always converges to a local optimum. Escaping requires changing multiple nodes simultaneously — a move 1-swap can never make. Iterated Greedy solves this by randomly destroying part of the solution and rebuilding from scratch, then polishing with one 1-swap pass.

Each iteration:

- Destroy: randomly remove destroyRate% of the selected set (shuffled, then take first removeCount).
- Rebuild: iterate through the pre-sorted ratio order and greedily re-add any conflict-free node.
- Polish: run oneSwapPass() + fillFreeNodes() on the rebuilt candidate.
- Update: if candidate beats global best → update best and reset destroyRate to 0.25. Sideways moves (same score) accepted to escape flat plateaus.

Adaptive destroy rate:

- Starts at 0.25 (25%) — small perturbations, stays close to current solution.
- Increases by 0.05 every 30 non-improving rounds — widens the search when stuck.
- Resets to 0.25 after any improvement — exploit the new region before exploring further.
- Exits when rate reaches 0.55 and still no improvement after 30 rounds — landscape is flat.

The ratio order is pre-sorted once before the loop and reused every rebuild, avoiding an O(N log N) sort on every iteration.

5. Complexity Analysis

Component	Time Complexity	Notes
greedyBuild()	O(N + M)	One pass over adjacency list
fillFreeNodes()	O(N + M)	Single scan; called after every change
oneSwapPass()	$O(N \cdot d^2)$ per pass	d = avg degree; fast on sparse graphs
IG rebuild (per iter)	O(N + M)	Pre-sorted order reused; no re-sort
Full run (5 min)	Hundreds of IG iterations	~8 s on N = 200,000 sparse graph

Space complexity: O(N + M) for the adjacency list, inSet[], conflictCnt[], deg[], and skill[].

Score overflow: Maximum possible score = 200,000 × 10^9 = 2×10^{14}, which exceeds int range (2×10^9). All score variables use long long (64-bit), which holds up to ~9.2×10^{18}.

6. Implementation Notes

Solution Struct

All solution state (inSet[], conflictCnt[], score) is encapsulated in a Solution struct with add(), remove(), canAdd(), fillFreeNodes(), and selected() methods. This keeps all invariants consistent and makes the local search and iterated greedy logic easy to reason about and debug.

Fixed Random Seed

mt19937 rng(42) — the Mersenne Twister with seed 42. Fixed seed means the program produces the same output on every run with the same input, which is essential for reproducibility .

Timer Design

std::chrono::steady_clock measures wall time (unaffected by system load or CPU frequency scaling). Phase 1 greedy seeds are capped at 60 s; local search at 260 s; iterated greedy at 275 s. The adaptive convergence exit ensures small inputs terminate in milliseconds without waiting out the full budget.

Correctness Invariants

- conflictCnt[u] is always exactly the number of selected neighbours of u.
- canAdd(u) returns true if and only if u is not selected and has no selected neighbour.

- add() and remove() update conflictCnt for every neighbour in adj[u], keeping the invariant.
- The output is always a valid independent set by construction — no explicit validation step needed.

---

7. Results

Measured on N = 200,000 nodes with sparse edge density (~0.0005%):

Strategy	Score vs Baseline	Runtime
Greedy only (ratio order)	Baseline	< 1 s
+ 1-swap local search	+~400 M above baseline	< 1 s
+ Iterated Greedy (full run)	+~590 M above greedy+swap	~8 s

The iterated greedy phase contributed approximately 590 million additional skill points on the N = 200,000 test compared to greedy + 1-swap alone, demonstrating the value of escaping local optima through the destroy-and-rebuild mechanism.

---

8. Test Cases

File	N	M	Description	Expected Score
test1.txt	5	4	Small tree-like graph	20
test2.txt	7	7	Odd cycle C_7 — classic NP structure	430
test3.txt	1	0	Single node edge case	999999999
test4.txt	4	6	Complete graph K_4 — one node max	400

tests/gen.py generates random inputs at any N and edge density for stress testing.

---

9. Build and Run

Compile

```
g++ -O2 -std=c++17 -Wall -o hackathon_squad main.cpp
```

Run

```
./hackathon_squad < tests/test1.txt
```

Run all tests

```
make tests
```

Stress test (N = 200,000)

```
python3 tests/gen.py 200000 0.000005 42 > big.txt
./hackathon_squad < big.txt
```

---

10. References

- Garey, M.R. and Johnson, D.S. — Computers and Intractability: A Guide to the Theory of NP-Completeness (1979)
- Feo, T.A. and Resende, M.G.C. — Greedy Randomized Adaptive Search Procedures, Journal of Global Optimization (1995)
- Resende, M.G.C. and Ribeiro, C.C. — GRASP: Greedy Randomized Adaptive Search Procedures (2016)
- CP-Algorithms — Maximum Independent Set: cp-algorithms.com
- Project repository: github.com/Nimit2504/Hackathon-Squad